
WebSocket

Du temps réel aux vulnérabilités : fonctionnement, attaques et défense

Gana Faye

Sécurité des applications web | 20 juillet 2026

TEMPS RÉEL

À la fin, vous saurez :

- ▶ expliquer pourquoi WebSocket existe et quand l'utiliser ;
- ▶ décrire l'ouverture, la connexion et les trames ;
- ▶ lire un échange dans le navigateur ou Burp Suite ;
- ▶ comprendre les principales attaques WebSocket ;
- ▶ tester et sécuriser une implémentation de bout en bout.

Idée fondamentale

Une requête HTTP ouvre un canal

PERSISTANT

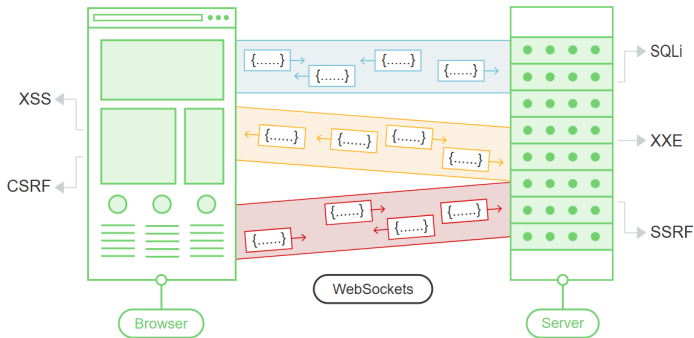
BIDIRECTIONNEL

dans lequel client et serveur peuvent parler à tout moment.

Cadre éthique

Les tests offensifs doivent viser uniquement une application autorisée ou un laboratoire.

Vue d'ensemble : où se trouve WebSocket ?



Lecture de l'image

Le navigateur et le serveur conservent un canal où circulent des messages dans les deux sens.

- ▶ Les messages sont souvent du JSON.
- ▶ La connexion reste ouverte.
- ▶ Les données reçues peuvent atteindre une base, un parseur ou une autre API.
- ▶ Les vulnérabilités web classiques restent donc possibles.

PARTIE 1

Comprendre WebSocket

Modèle HTTP

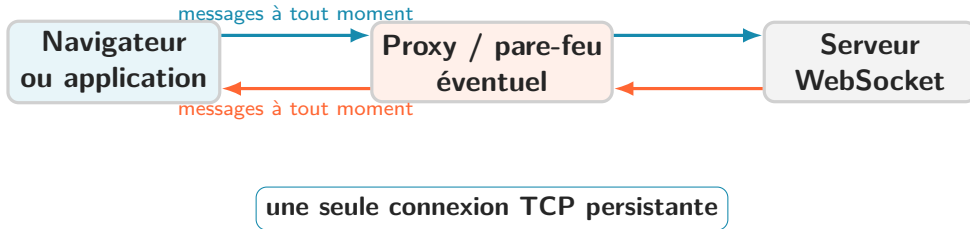
Le client demande, puis le serveur répond.
Le serveur ne peut normalement pas envoyer spontanément une nouvelle donnée.



Pour simuler le temps réel

- ▶ **Polling** : requêtes répétées, même sans nouveauté.
- ▶ **Long polling** : la réponse attend un événement, puis le client recommence.
- ▶ **SSE** : flux serveur vers client, unidirectionnel.

WebSocket réduit ces allers-retours lorsque les deux côtés échangent fréquemment.



Full-duplex

Les deux côtés envoient indépendamment.

Faible surcharge

Après l'ouverture, les trames sont compactes.

Événementiel

Un message arrive dès qu'un événement survient.

WebSocket n'est pas « un autre Internet »

	HTTP / REST	WebSocket
Relation	requête puis réponse	canal persistant full-duplex
Déclencheur	principalement le client	client ou serveur
En-têtes	à chaque requête HTTP	surtout pendant l'ouverture
Format applicatif	JSON, HTML, fichiers, etc.	texte ou binaire ; protocole à définir
État	interactions souvent indépendantes	connexion naturellement stateful
Bon usage	CRUD, pages, ressources	chat, jeu, collaboration, télémétrie

À ne pas confondre

WebSocket est un **transport de messages**. Il ne définit ni le schéma JSON, ni les permissions, ni la logique métier de l'application.

ws://

Canal non chiffré, comparable à HTTP. Un intermédiaire réseau peut observer ou modifier le trafic.

```
ws://example.test/chat
```

wss://

WebSocket protégé par TLS, comparable à HTTPS : confidentialité, intégrité et authentification du serveur.

```
wss://example.test/chat
```

Règle pratique

En production, utiliser wss://. Une page HTTPS doit ouvrir un canal sécurisé pour éviter le contenu mixte et l'interception.

PARTIE 2

Ouverture et fonctionnement interne

Étape 1 : le handshake HTTP

```
GET /chat HTTP/1.1
Host: example.test
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Version: 13
Sec-WebSocket-Key: dGh1IHhnbXBsZQ==
Origin: https://example.test
Cookie: session=abc123
```

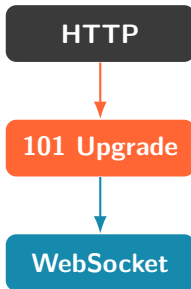
- 1 Le navigateur initie une requête HTTP.
- 2 Upgrade demande le changement de protocole.
- 3 La clé aléatoire participe à la validation du serveur.
- 4 Origin indique la page à l'origine de l'ouverture.
- 5 Cookies et jetons peuvent authentifier l'utilisateur.

Étape 2 : le serveur accepte l'upgrade

```
HTTP/1.1 101 Switching Protocols  
Upgrade: websocket  
Connection: Upgrade  
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYG...
```

Code 101

Le serveur accepte de quitter l'échange HTTP classique. La connexion sous-jacente est conservée pour les trames WebSocket.



La valeur Sec-WebSocket-Accept prouve que le serveur a compris la clé. Elle **n'authentifie pas** l'utilisateur.

Après le handshake : des trames



- ▶ **FIN** : dernier fragment du message.
- ▶ **Opcode** : texte, binaire, fermeture, ping ou pong.
- ▶ **Payload** : contenu applicatif.
- ▶ Les trames client vers serveur sont masquées.
- ▶ Le masquage protège certains intermédiaires ; ce n'est **pas** du chiffrement.
- ▶ TLS reste nécessaire avec `wss://`.

Types de trames et cycle de vie

Opcode	Type	Rôle
0x1	Texte	chaîne UTF-8, souvent du JSON
0x2	Binaire	image, audio, format sérialisé ou données métier
0x0	Continuation	fragment suivant d'un message long
0x8	Close	fermeture propre avec code et raison éventuelle
0x9 / 0xA	Ping / Pong	maintien et contrôle de la connexion



Exemple applicatif : un chat

```
const ws = new WebSocket(
  "wss://example.test/chat"
);

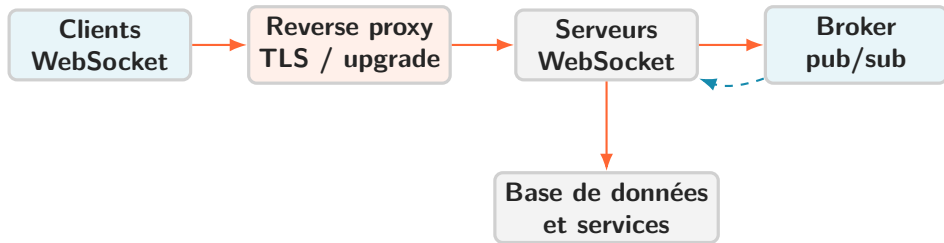
ws.onopen = () => ws.send(JSON.stringify({
  type: "join",
  room: "cours-web"
}));

ws.onmessage = event => {
  const message = JSON.parse(event.data);
  afficher(message);
};
```

```
// Message client -> serveur
{"type":"message",
 "room":"cours-web",
 "text":"Bonjour !"}

// Message serveur -> clients
{"type":"message.created",
 "author":"Awa",
 "text":"Bonjour !",
 "sentAt":"12:06:42Z"}
```

Le serveur doit valider le **type**, la **taille**, les **champs**, l'**identité** et l'**autorisation** de chaque message.



- ▶ Le proxy doit conserver les en-têtes d'upgrade et des délais adaptés.
- ▶ Plusieurs serveurs nécessitent souvent un broker pour diffuser un événement.
- ▶ Chaque connexion consomme des ressources : mémoire, socket, état et tâches.
- ▶ Une reconnexion doit restaurer proprement l'état sans dupliquer les actions.

PARTIE 3

Observer et tester les échanges

DevTools du navigateur

Onglet *Network* → filtre *WS* → connexion → *Messages*.

- ▶ vérifier l'URL et le code 101 ;
- ▶ lire les messages entrants et sortants ;
- ▶ observer fermeture et reconnexion.

Burp Proxy

Proxy → *WebSockets history*. Les messages peuvent être interceptés et modifiés dans un laboratoire.

- ▶ modifier un champ JSON ;
- ▶ renvoyer un message avec Repeater ;
- ▶ comparer deux comptes et deux rôles.

Important

Tester le handshake **et** les messages : une connexion correctement authentifiée peut contenir des actions mal autorisées.



Question constante

Si je modifie ce message, le serveur **recalcule-t-il** l'identité et l'autorisation, ou fait-il confiance au client ?

PARTIE 4

Comprendre les attaques WebSocket

Carte des principales vulnérabilités

Famille	Erreur	Impact possible
Authentification faible	handshake accepté sans contrôle robuste	usurpation, accès au canal
CSWSH	origine non vérifiée avec cookies	actions ou fuite au nom de la victime
Autorisation absente	confiance dans userId, roomId	IDOR, élévation de privilèges
Injection	message concaténé dans HTML, SQL, XML, commande	XSS, SQLi, XXE, RCE
Logique métier	ordre, quantité ou état manipulable	fraude, double action
Disponibilité	taille, fréquence ou connexions illimitées	déni de service

Principe

WebSocket ne crée pas toutes ces failles : il devient un nouveau point d'entrée vers la même logique applicative.

Attaque 1 : manipulation d'un message

```
// Message normal
{"action": "getProfile",
 "userId": 1042}

// Message modifié
{"action": "getProfile",
 "userId": 1043}
```

- 1 L'utilisateur ouvre une connexion légitime.
- 2 Il intercepte son propre message.
- 3 Il change un identifiant ou un rôle.
- 4 Le serveur utilise la valeur sans vérifier la propriété.
- 5 Les données d'un autre compte sont renvoyées.

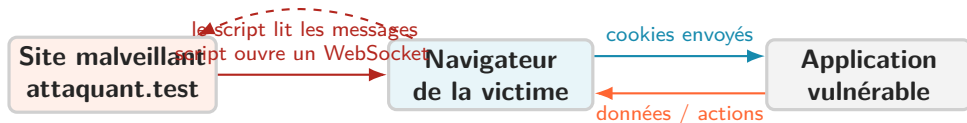
Cause réelle

Une IDOR / BOLA : le serveur confond **paramètre fourni** et **autorisation accordée**.

Correction

Déduire l'identité de la session serveur et vérifier l'accès à chaque objet.

Attaque 2 : Cross-Site WebSocket Hijacking



Conditions habituelles

Authentification par cookie + handshake sans jeton imprévisible + absence de validation stricte de Origin.

Effet

Le navigateur de la victime ouvre un canal authentifié depuis un autre site ; l'attaquant peut envoyer des commandes et parfois lire les réponses.

Scénario CSWSH, étape par étape

- 1 La victime est déjà connectée à `https://bank.example`.
- 2 Elle visite une page contrôlée par l'attaquant.
- 3 Cette page exécute `new WebSocket("wss://bank.example/ws")`.
- 4 Le navigateur joint les cookies autorisés au handshake.
- 5 Le serveur ne vérifie pas que `Origin` correspond à son site.
- 6 La connexion est acceptée avec l'identité de la victime.
- 7 Le script malveillant envoie une action ou exfiltre une réponse.

Mauvais contrôle

Autoriser toute origine, une sous-chaîne comme `example.com`, ou accepter un `Origin: null` sans justification.

Bon contrôle

Comparaison exacte avec une liste d'origines HTTPS autorisées, puis jeton anti-CSRF lié à la session si nécessaire.

Attaque 3 : XSS transportée par WebSocket

```
{"type":"message",  
  "text":"<img src=x onerror=...>"}
```

Chaîne vulnérable

Le serveur relaie le texte, puis le navigateur l'insère avec `innerHTML`. Le message devient du code HTML actif.

```
// Dangereux  
chat.innerHTML += message.text;  
  
// Texte non interprété comme HTML  
const line = document.createElement("p");  
line.textContent = message.text;  
chat.appendChild(line);
```

Défense

Encodage contextuel, API DOM sûres, validation, CSP et assainissement si le HTML est réellement nécessaire.

Attaque 4 : injections côté serveur

SQLi

Un champ JSON est concaténé dans une requête SQL au lieu d'utiliser des paramètres.

Commande / template

Une valeur contrôle une commande système, un nom de fichier ou une expression de template.

XXE / désérialisation

Le serveur accepte XML ou un objet complexe avec un parseur dangereux.

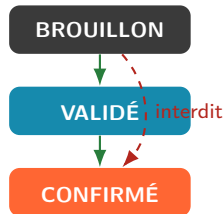


Défense

Schéma strict, requêtes paramétrées, API sûres, privilèges minimaux et aucune interprétation implicite des données reçues.

Exemples

- ▶ envoyer confirm avant valide ;
- ▶ rejouer deux fois une opération ;
- ▶ modifier prix, quantité ou destinataire ;
- ▶ publier dans une salle non rejointe ;
- ▶ utiliser un message réservé à l'administrateur.



Cause

Le serveur suppose que l'interface impose le bon ordre. Or un client peut fabriquer n'importe quel message.

- ▶ trop de connexions simultanées ;
- ▶ messages trop fréquents ou trop volumineux ;
- ▶ fragmentation abusive ;
- ▶ compression coûteuse ;
- ▶ connexions inactives jamais fermées ;
- ▶ diffusion d'un message vers des milliers de clients.

Particularité

Une connexion persistante immobilise des ressources plus longtemps qu'une requête HTTP courte.

Contrôles nécessaires

- ▶ quotas par IP, compte et connexion ;
- ▶ taille et fréquence maximales ;
- ▶ délais d'inactivité et ping/pong ;
- ▶ backpressure et files bornées ;
- ▶ limite de connexions et surveillance.

Risque	Question de test
Données sensibles	Les secrets passent-ils en clair, dans les journaux ou dans des erreurs ?
Session expirée	Une connexion ouverte reste-t-elle active après déconnexion ou révocation ?
Sous-protocole	Le serveur accepte-t-il un Sec-WebSocket-Protocol inattendu ?
Compression	permessage-deflate est-il utile, limité et évalué contre les fuites latérales ?
Reconnexion	Peut-elle rejouer une opération ou contourner une limite ?
Proxy / cache	Les équipements traitent-ils correctement l'upgrade sans mettre en cache de données ?
Journalisation	Les événements permettent-ils d'attribuer une action à un compte et une connexion ?

PARTIE 5

Sécuriser une application WebSocket

Transport et origine

- ▶ imposer `wss://` ;
- ▶ vérifier exactement `Origin` ;
- ▶ refuser origine absente ou `null`, sauf besoin documenté ;
- ▶ configurer correctement proxy et TLS.

Authentification

- ▶ session ou jeton court et validé côté serveur ;
- ▶ jeton anti-CSRF lié à la session si cookies ;
- ▶ ne pas placer un secret durable dans l'URL ;
- ▶ limiter sous-protocoles et extensions.

Attention

CORS ne protège pas automatiquement WebSocket. La validation de `Origin` doit être implémentée par le serveur WebSocket.



- ▶ Liste blanche des types de messages et des champs attendus.
- ▶ Refus des propriétés inconnues si elles peuvent modifier le comportement.
- ▶ Autorisation recalculée, même si l'interface n'affiche pas l'action.
- ▶ Réponses minimales : ne pas diffuser plus de données que nécessaire.

Gérer la session pendant toute la connexion

Au démarrage

Authentifier la connexion, associer un identifiant serveur et enregistrer compte, rôle, origine et heure.

Lors d'un changement

Déconnecter ou réauthentifier après déconnexion, révocation, changement de mot de passe ou de rôle.

Pendant les échanges

Réévaluer les permissions sensibles et appliquer quotas, expiration, limites de taille et délais.

À la fermeture

Libérer abonnements, verrous, files et ressources ; journaliser la cause et le code de fermeture.

Contrôle	Résultat attendu
Transport	uniquement wss://, TLS valide
Origine	liste blanche exacte, aucun contournement par suffixe ou sous-chaîne
Authentification	jeton/session valide, expiration et révocation effectives
Autorisation	test par action, objet, rôle, salle et tenant
Entrées	schéma strict, types et tailles limités
Sorties	encodage selon le contexte, aucune donnée excessive
Résilience	quotas, timeout, backpressure, limite de connexions
Traçabilité	connexion, compte, type d'action et erreur journalisés sans secret

Mini-plan de test avec deux comptes

- 1 Ouvrir deux sessions : utilisateur A et utilisateur B.
- 2 Comparer handshake, cookies, jetons et messages.
- 3 Rejouer un message de A dans la connexion de B.
- 4 Modifier identifiant d'objet, salle, tenant et type d'action.
- 5 Tester une origine externe dans un laboratoire.
- 6 Envoyer champs manquants, types incorrects, message vide et taille limite.
- 7 Déconnecter A, révoquer sa session, puis vérifier que son canal cesse d'agir.
- 8 Confirmer les codes d'erreur, les journaux et l'absence de données sensibles.

Critère de réussite

Le serveur refuse proprement les messages invalides, sans fuite d'information et sans modifier l'état métier.

PARTIE 6

Synthèse

Choisir WebSocket ou une autre solution ?

Besoin	HTTP / REST	SSE	WebSocket
CRUD ponctuel	excellent	peu adapté	souvent excessif
Flux serveur seul	polling possible	excellent	adapté
Échanges fréquents dans les deux sens	coûteux	insuffisant	excellent
Cache HTTP	naturel	limité	non applicable aux trames
Simplicité opérationnelle	forte	moyenne	plus faible
Exemples	formulaire, API	notifications, cours	chat, jeu, édition live

Décision

Choisir WebSocket quand le besoin bidirectionnel et la faible latence justifient la gestion supplémentaire des connexions, de l'état et de la sécurité.

- 1 WebSocket commence par un **handshake HTTP**, puis utilise un canal persistant.
- 2 Il transporte des trames texte ou binaires, mais la logique des messages appartient à l'application.
- 3 `wss://` protège le transport ; il ne remplace ni authentification ni autorisation.
- 4 Chaque message est une nouvelle entrée non fiable : valider, autoriser, limiter et encoder.
- 5 Les attaques majeures viennent de l'origine non vérifiée, de la confiance dans le client, des injections et de l'absence de limites.

Une connexion authentifiée ne signifie pas que tous ses messages sont autorisés.

- ▶ IETF, *RFC 6455 — The WebSocket Protocol*.
- ▶ IETF, *RFC 8441 — Bootstrapping WebSockets with HTTP/2*.
- ▶ IETF, *RFC 9220 — Bootstrapping WebSockets with HTTP/3*.
- ▶ OWASP, *WebSocket Security Cheat Sheet*.
- ▶ PortSwigger Web Security Academy, *WebSockets vulnerabilities*.
- ▶ MDN Web Docs, *The WebSocket API*.

Pour pratiquer légalement

Utiliser un laboratoire local, une application volontairement vulnérable ou les exercices WebSocket de la Web Security Academy.

WEBSOCKET

Questions ?

Comprendre le protocole, observer les messages, vérifier chaque autorisation.

Gana Faye | Sécurité des applications web